

Distance Metric Learning for Large Margin Nearest Neighbor Classification

王哲涛 叶翔 贺嘉言 钱周越

数学科学学院
浙江大学

2025 年 5 月 6 日

Outline

Outline

Introduction

Background

Model

Results

References

Introduction



The Problem

- k-Nearest Neighbor (kNN) Classification
 - Simple, effective rule: Classify by majority label of k closest training examples.
 - Performance depends crucially on the distance metric used.
- Default: Euclidean Distance
 - Ignores statistical regularities in data.
 - Example: Same metric for face age vs. gender classification is suboptimal.
- Motivation: Improve kNN accuracy by learning an appropriate distance metric from labeled data.
 - Previous work showed simple linear transformations help.
 - We aim to learn a **Mahalanobis distance metric**.

Our Approach - LMNN

- Intuition: A kNN classifier is accurate if:
 - An example's k -nearest neighbors belong to the same class.
 - Examples from different classes are separated by a large margin.
- LMNN aims to learn a linear transformation (Mahalanobis metric) that achieves these properties.
- Achieved by minimizing a loss function:
 - Penalizes large distances between desired neighbors.
 - Penalizes small distances between differently labeled examples (using a hinge loss for a margin).
- This yields a convex optimization problem.

Background



Distance Metrics

- Definition: A mapping $D : \mathcal{X} \times \mathcal{X} \rightarrow \mathfrak{R}_0^+$ is a metric if it satisfies:
 - ① Triangular inequality: $D(\vec{x}_i, \vec{x}_j) + D(\vec{x}_j, \vec{x}_k) \geq D(\vec{x}_i, \vec{x}_k)$
 - ② Non-negativity: $D(\vec{x}_i, \vec{x}_j) \geq 0$
 - ③ Symmetry: $D(\vec{x}_i, \vec{x}_j) = D(\vec{x}_j, \vec{x}_i)$
 - ④ Distinguishability: $D(\vec{x}_i, \vec{x}_j) = 0 \iff \vec{x}_i = \vec{x}_j$
- Pseudometric: Satisfies 1-3 but not necessarily 4.

Mahalanobis Distance

- A family of metrics/pseudometrics derived from a linear transformation $\vec{x}' = \mathbf{L}\vec{x}$ followed by Euclidean distance.

- Squared distance:

$$D_{\mathbf{L}}(\vec{x}_i, \vec{x}_j) = \|\mathbf{L}(\vec{x}_i - \vec{x}_j)\|_2^2$$

- Can be expressed in terms of a positive semidefinite matrix $\mathbf{M}$:

$$\mathbf{M} = \mathbf{L}^\top \mathbf{L} \quad \implies \quad \mathcal{D}_{\mathbf{M}}(\vec{x}_i, \vec{x}_j) = (\vec{x}_i - \vec{x}_j)^\top \mathbf{M} (\vec{x}_i - \vec{x}_j)$$

- Euclidean distance is a special case when $\mathbf{M}$ is the identity matrix.
- Metric learning goal: Estimate $\mathbf{L}$ or $\mathbf{M}$ from labeled data $\{(\vec{x}_i, y_i)\}_{i=1}^n$.

Review of Previous Approaches

- Broadly classified into three categories:
 - ① Eigenvector Methods (based on second-order statistics).
 - ② Convex Optimizations (over positive semidefinite matrices).
 - ③ Fully Supervised Algorithms (directly optimizing kNN criteria).
- These approaches provide context and motivation for LMNN.

Eigenvector Methods

Principal Component Analysis (PCA)

- Goal: Compute $\mathbf{L}$ that projects data into a variance-maximizing subspace.
- Method: Unsupervised. Based on input covariance matrix $\mathbf{C}$.

$$\mathbf{C} = \frac{1}{n} \sum_{i=1}^n (\vec{x}_i - \vec{\mu})(\vec{x}_i - \vec{\mu})^\top$$

- Optimization:

$$\max_{\mathbf{L}} \text{Tr}(\mathbf{L}^\top \mathbf{C} \mathbf{L}) \quad \text{subject to: } \mathbf{L} \mathbf{L}^\top = \mathbf{I}$$

- Solution: Rows of $\mathbf{L}$ are leading eigenvectors of $\mathbf{C}$.

Eigenvector Methods

Linear Discriminant Analysis (LDA)

- Goal: Compute $\mathbf{L}$ that maximizes the ratio of between-class to within-class variance.
- Method: Supervised. Based on between-class ($\mathbf{C}_b$) and within-class ($\mathbf{C}_w$) covariance matrices.

$$\mathbf{C}_b = \frac{1}{C} \sum_{c=1}^C \vec{\mu}_c \vec{\mu}_c^\top \quad \mathbf{C}_w = \frac{1}{n} \sum_{c=1}^C \sum_{i \in \Omega_c} (\vec{x}_i - \vec{\mu}_c) (\vec{x}_i - \vec{\mu}_c)^\top$$

- Optimization:

$$\max_{\mathbf{L}} \text{Tr} \left(\frac{\mathbf{L}^\top \mathbf{C}_b \mathbf{L}}{\mathbf{L}^\top \mathbf{C}_w \mathbf{L}} \right) \quad \text{subject to: } \mathbf{L} \mathbf{L}^\top = \mathbf{I}$$

- Solution: Rows of $\mathbf{L}$ are leading eigenvectors of $\mathbf{C}_w^{-1} \mathbf{C}_b$.

Eigenvector Methods

Relevant Component Analysis (RCA)

- Goal: Compute $\mathbf{L}$ that whitens data w.r.t. averaged within-"chunklet" covariance.
- Method: Supervised, uses "chunklet" information (subsets of a class). Ω_ℓ is the set of indices in chunklet ℓ .

$$\mathbf{C}_w = \frac{1}{n} \sum_{l=1}^L \sum_{i \in \Omega_l} (\vec{x}_i - \vec{\mu}_l) (\vec{x}_i - \vec{\mu}_l)^\top$$

- Linear transformation $\vec{x}_i \rightarrow \mathbf{L} \vec{x}_i$ with $\mathbf{L} = \mathbf{C}_w^{-1/2}$.
- Often used after PCA.

Convex Optimization

Mahalanobis Metric for Clustering (MMC)

- Goal: Learn $\mathbf{M}$ for clustering, minimizing distances between similarly labeled inputs, maximizing distances between differently labeled inputs.
- Comparison to LDA: Similar goal, but formulated as a convex optimization over $\mathbf{M}$ instead of an eigenvalue problem for $\mathbf{L}$.
- Formulation: Convex optimization over $\mathbf{M}$. Uses binary association matrix $y_{ij} = 1$ if $y_i = y_j$, 0 otherwise.

Maximize $\sum_{ij} (1 - y_{ij}) \sqrt{\mathcal{D}_{\mathbf{M}}(\vec{x}_i, \vec{x}_j)}$ **subject to:**

(1) $\sum_{ij} y_{ij} \mathcal{D}_{\mathbf{M}}(\vec{x}_i, \vec{x}_j) \leq 1$

(2) $\mathbf{M} \succeq 0$.

Convex Optimization

Pseudometric Online Learning Algorithm (POLA)

- Goal: Perceptron-like online learning of $\mathbf{M}$ with an explicit margin. Shrink distances for same-label pairs, expand for different-label pairs.
- Comparison to LDA/MMC: Also aims to separate classes, but introduces a finite margin and can work online.
- Online Setting: Receives tuples $(\vec{x}_t, \vec{x}_t', y_t)$ with $y_t \in \{1, -1\}$
- Margin Constraint: Aims for similarly labeled pairs $\leq b - 1$ distance, differently labeled pairs $\geq b + 1$ distance.

$$y_t [b - \mathcal{D}_{\mathbf{M}}(\vec{x}_t, \vec{x}_t')] \geq 1$$

- Method: Updates $\mathbf{M}$ and b online using an alternating projection algorithm to correct margin violations. $\mathbf{M}$ remains PSD after updates.

Neighborhood Component Analysis (NCA)

- Goal: Learn a Mahalanobis metric $\mathbf{M}$ (or $\mathbf{L}$) specifically to improve kNN classification accuracy.
- Approach: Minimize the expected leave-one-out (LOO) classification error using a **stochastic** variant of kNN.
- Stochastic kNN: Probability p_{ij} of $\vec{x}_j$ being chosen as a reference for $\vec{x}_i$ is based on a softmax over distances $\mathcal{D}_{\mathbf{L}}(\vec{x}_i, \vec{x}_j)$:

$$p_{ij} = \begin{cases} \frac{\exp(-\|\mathbf{L}\vec{x}_i - \mathbf{L}\vec{x}_j\|^2)}{\sum_{k \neq i} \exp(-\|\mathbf{L}\vec{x}_i - \mathbf{L}\vec{x}_k\|^2)} & \text{if } i \neq j \\ 0 & \text{if } i = j. \end{cases}$$

(No fixed k parameter; scale of $\mathbf{L}$ controls neighborhood size).

Neighborhood Component Analysis (NCA)(cont.)

- Objective: Expected LOO classification accuracy on training data. Minimizing error is maximizing expected accuracy.

$$\mathcal{E}_{\text{NCA}} = 1 - \frac{1}{n} \sum_{ij} p_{ij} y_{ij}$$

- The objective function $\mathcal{E}_{\text{NCA}}$ is continuous and differentiable w.r.t. $\mathbf{L}$ (or $\mathbf{M}$), unlike deterministic kNN LOO error. This allows optimization via gradient descent.
- The objective function is non-convex. Optimization can get stuck in spurious local minima, results depend on initialization.

Model



Building on Distance Metric Learning

- Goal: Learn a Mahalanobis distance metric

$$\mathcal{D}_{\mathbf{M}}(\vec{x}_i, \vec{x}_j) = (\vec{x}_i - \vec{x}_j)^{\top} \mathbf{M} (\vec{x}_i - \vec{x}_j) \text{ where } \mathbf{M} \succeq 0.$$

- $\mathbf{M}$ is learned from labeled data.
- Builds on existing approaches:
 - **MMC**: Formulate estimation as convex optimization ($\mathbf{M} \succeq 0$).
 - **POLA**: Maximize classification margin.
 - **NCA**: Specifically designed to improve k NN classification.

Intuition for Robust k NN

Two simple intuitions guide the model:

- ① **Neighborhood Homogeneity:** Each training input $\vec{x}_i$ should share the same label y_i as its k nearest neighbors (in the learned metric).
- ② **Class Separation:** Training inputs with different labels should be widely separated.

These are balanced by competing terms in the loss function:

- Term 1: Penalizes large distances between nearby inputs with the same label. (Pull)
- Term 2: Penalizes small distances between inputs with different labels. (Push)

Key Terminology

- **Target Neighbors** ($j \rightsquigarrow i$):
 - Training inputs $\vec{x}_j$ that we desire to be closest to $\vec{x}_i$.
 - Fixed *a priori* (e.g., k nearest neighbors of $\vec{x}_i$ with the same class label y_i using Euclidean distance).
- **Importors**:
 - Training inputs $\vec{x}_l$ with a different label ($y_l \neq y_i$), $\vec{x}_l$ is an importor if $\|\mathbf{L}(\vec{x}_i - \vec{x}_l)\|^2 \leq \|\mathbf{L}(\vec{x}_i - \vec{x}_j)\|^2 + 1$.
 - Goal of learning: minimize the number of importors.
- **Margin**:
 - Maintain a large distance between importors and the perimeters defined by target neighbors.
 - Improves robustness to noise.

Schematic Illustration

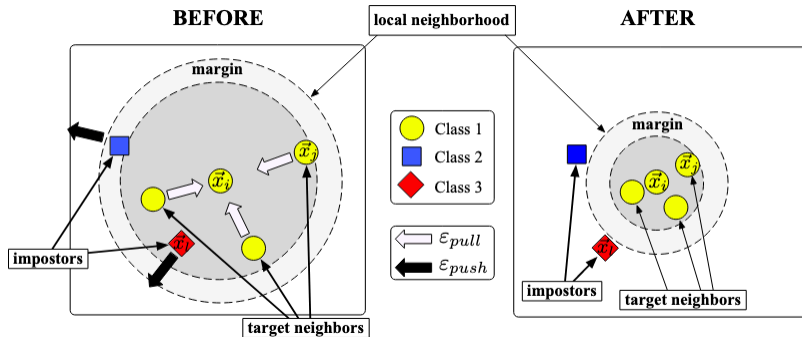


Figure: LMNN Learning Illustration

Loss Function

- The loss function consists of two parts:

$$\varepsilon(\mathbf{L}) = (1 - \mu)\varepsilon_{pull}(\mathbf{L}) + \mu\varepsilon_{push}(\mathbf{L}),$$

where $\mu \in [0, 1]$ is a balancing parameter, typically set to 0.5.

- $\varepsilon_{pull}(\mathbf{L}) = \sum_{j \rightsquigarrow i} \|\mathbf{L}(\vec{x}_i - \vec{x}_j)\|^2$, which penalizes large distances between each input and its target neighbors, attracting target neighbors closer.
- $\varepsilon_{push}(\mathbf{L}) = \sum_{i, j \rightsquigarrow i} \sum_l (1 - y_{il}) [1 + \|\mathbf{L}(\vec{x}_i - \vec{x}_j)\|^2 - \|\mathbf{L}(\vec{x}_i - \vec{x}_l)\|^2]_+$, which uses hinge loss to penalize small distances between differently-labeled samples, pushing impostors away.

Local vs. Global Distances

- LMNN's focus on pulling only *target neighbors* (local) is a key distinction.
- Many previous methods (LDA, RCA, MCC) try to shrink distances between *all* examples in the same class (global).

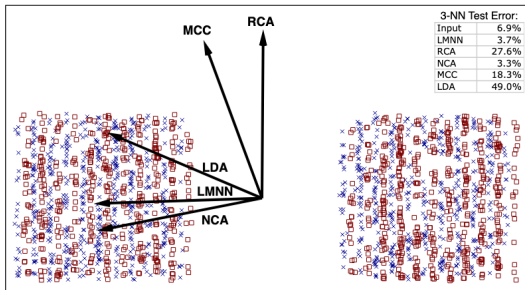


Figure: Toy Data Example

Convex Optimization

Problem with Optimization in $\mathbf{L}$:

- The loss function $\varepsilon(\mathbf{L})$ is *not convex* in $\mathbf{L}$.
- Gradient descent in $\mathbf{L}$ is prone to being trapped in spurious local minima.

Solution: Reformulate as Convex Optimization over $\mathbf{M}$

- Use the variable $\mathbf{M} = \mathbf{L}^\top \mathbf{L}$, the constraint $\mathbf{M} \succeq 0$ is convex.
- Loss Function in terms of $\mathbf{M}$:

$$\begin{aligned} \varepsilon(\mathbf{M}) = & (1 - \mu) \sum_i \sum_{j \rightsquigarrow i} \mathcal{D}_{\mathbf{M}}(\vec{x}_i, \vec{x}_j) \\ & + \mu \sum_i \sum_{j \rightsquigarrow i} \sum_l (1 - y_{il}) [1 + \mathcal{D}_{\mathbf{M}}(\vec{x}_i, \vec{x}_j) - \mathcal{D}_{\mathbf{M}}(\vec{x}_i, \vec{x}_l)]_+. \end{aligned}$$

Convex Optimization(cont.)

Formulation as a Semidefinite Program (SDP):

LMNN SDP

Minimize:

$$(1 - \mu) \sum_{i, j \rightsquigarrow i} (\vec{x}_i - \vec{x}_j)^\top \mathbf{M} (\vec{x}_i - \vec{x}_j) + \mu \sum_{i, j \rightsquigarrow i, l} (1 - y_{il}) \xi_{ijl}$$

subject to:

$$(1) (\vec{x}_i - \vec{x}_l)^\top \mathbf{M} (\vec{x}_i - \vec{x}_l) - (\vec{x}_i - \vec{x}_j)^\top \mathbf{M} (\vec{x}_i - \vec{x}_j) \geq 1 - \xi_{ijl}, \quad \forall i, j \rightsquigarrow i, l$$

$$(2) \xi_{ijl} \geq 0, \quad \forall i, j \rightsquigarrow i, l$$

$$(3) \mathbf{M} \succeq 0$$

Convex Optimization(cont.)

Solving the SDP:

- LMNN exploits the **sparsity** of active constraints: Most impostors $\vec{x}_l$ are far away, so their hinge loss is 0 ($\xi_{ijl} = 0$). Only triplets violating or near violating the margin contribute positively to the objective and constraints.
- A special-purpose solver is implemented, based on a combination of:
 - Sub-gradient descent in $\mathbf{L}$ and $\mathbf{M}$.
 - Projection of updates onto the positive semidefinite cone for $\mathbf{M}$.
 - Uses alternating projection algorithms (provably convergent).
- Faster than generic solvers by focusing on potentially active constraints.

Energy Based Classification

Procedure for $\vec{x}_t$:

- ① For each possible class label y_t :
- ② Temporarily assign y_t to $\vec{x}_t$.
- ③ Identify k target neighbors for $\vec{x}_t$ among the training data assuming this label y_t .
- ④ Compute an "energy" score using the learned $\mathbf{M}$, considering $\vec{x}_t$ and its hypothetical target neighbors, as well as potential impostors.
- ⑤ The predicted label is the one that minimizes this energy score.

Energy Based Classification(cont.)

- Calculate energy score

$$\begin{aligned}
 E(y_t) = & (1 - \mu) \sum_{j \rightsquigarrow t} \mathcal{D}_{\mathbf{M}}(\vec{x}_t, \vec{x}_j) + \mu \sum_{j \rightsquigarrow t, l} (1 - y_{tl}) [1 + \mathcal{D}_{\mathbf{M}}(\vec{x}_t, \vec{x}_j) - \mathcal{D}_{\mathbf{M}}(\vec{x}_t, \vec{x}_l)]_+ \\
 & + \mu \sum_{i, j \rightsquigarrow i} (1 - y_{it}) [1 + \mathcal{D}_{\mathbf{M}}(\vec{x}_i, \vec{x}_j) - \mathcal{D}_{\mathbf{M}}(\vec{x}_i, \vec{x}_t)]_+
 \end{aligned}$$

and the classification rule is

$$y_t = \operatorname{argmin}_{y_t} \{E(y_t)\}$$

- It often leads to significant further improvements in test error rates compared to simply using the learned $\mathbf{M}$ with standard kNN.

Results



Summary of Experimental Results

• Experimental Setup

- Evaluated LMNN classification using nine datasets. Some high - dimensional datasets were pre - processed by PCA for dimensionality reduction.
- Fixed parameters $k = 3$ and $\mu = 0.5$. For most datasets, results were obtained by averaging over multiple runs with 70/30 random splits, except for Isolet and MNIST datasets.

• Result Trends

- Mahalanobis distance improved the accuracy of kNN classification, and the energy - based decision rule further enhanced the performance.
- Combining LMNN with PCA for dimensionality reduction was more effective, and LMNN showed greater advantages on larger datasets.

Summary of Experimental Results

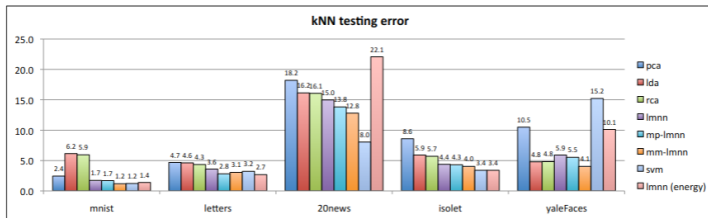
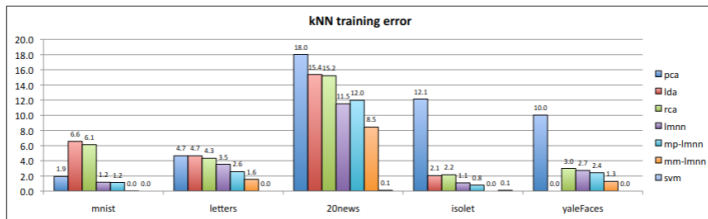
- **Comparison with Other Methods**

- The energy - based LMNN had similar results to multi - class SVMs, and the multi - metric LMNN was superior on some large datasets.
- LMNN outperformed methods like RCA, LDA, and NCA on the four largest datasets.

- **Small Datasets with Few Classes (4.1)**

- On small datasets such as wine, iris, and bal, LMNN classification showed some improvement, but it was difficult to assess the significance, and these were not the most suitable scenarios for LMNN.

Experimental Results



Face Recognition Results

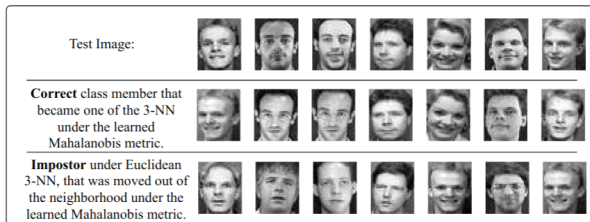


Figure: Test images from the Olivetti face recognition data set (top row). The middle row shows images from the same class that were among the 3-NN under the learned Mahalanobis metric (after training) but not among the original 3-NN under the Euclidean metric (before training). The bottom row shows impostors under the Euclidean metric that were no longer inside the local neighborhoods under the Mahalanobis metric.

Face Recognition Results

- **Olivetti Face Dataset**

- 400 grayscale images of 40 subjects (10 poses/subject).
- Downsampled to 38×31 pixels, projected to 200 - eigenface subspace by PCA.
- 7 - train, 3 - test images per subject. LMNN improved accuracy, altered $k = 3$ NN.

- **(Extended) Yale Face Dataset**

- 2414 frontal images of 38 subjects under extreme illumination.
- Pre - processed: downsampled, projected to 200 PCs, discarded 1st 5 eigenvectors.
- Results from 10 runs of 70/30 splits. LMNN outperformed Euclidean and SVMs.

Spoken Letter Recognition Results

- **Isolet Dataset**

- From UCI Machine Learning Repository, has 6238 examples for 26 letter classes.
- Original dimension: 617. Projected onto 172 principal components (covering 95% variance).

- **Experimental Results**

- Dietterich and Bakiri achieved test error rates of 4.2% ($26 - \text{output units}$) and 3.3% ($30 - \text{bit error} - \text{correcting code}$) using nonlinear backpropagation networks.
- LMNN with energy - based classification obtained a 3.4% test error rate.

- **Result Analysis**

- LMNN is an effective method for spoken letter recognition.
- Although not surpassing the best results, it shows potential in complex speech data classification.

Letter Recognition Results

- **Dataset Description**

- Taken from UCI Machine Learning Repository.
- Contains randomly distorted images of 26 English letters in 20 fonts.
- Features include 16 attributes like height, width, and axis correlations.

- **Experimental Results**

- LMNN with energy - based classification significantly outperformed other variants of kNN classification on this dataset.

- **Advantage Analysis**

- LMNN learns the Mahalanobis distance metric, effectively capturing complex data relationships.
- Its loss function design makes similar samples cluster closely and different - class samples separate well, enhancing adaptability and accuracy in letter recognition.

Letter Recognition Results

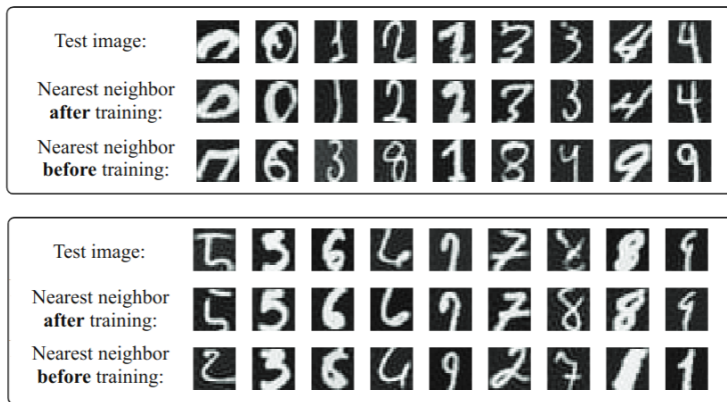


Figure: Images from the *MNIST* data set, along with nearest neighbors before and after training.

Text Categorization Results

- **Dataset**

- 20 - newsgroups: 18828 articles from 20 newsgroups.
- Tokenized, represented by 20,000 - word vectors, reduced to 200 PCs.

- **Results**

- LMNN improved over kNN with Euclidean/PCA. Error rate: 14.98% vs 48.57%/18.22%.
- But multi - class SVM was better (8.0% error with linear kernel, 20,000D input).

- **Analysis**

- LMNN used data features, but SVM was more effective for high - D text.
- Future research could optimize LMNN for text, e.g., new distance metrics.

Handwritten Digit Recognition Results

- **Dataset Description**

- The MNIST dataset of handwritten digits is widely benchmarked.
- Original 28×28 grayscale images were deskewed and projected onto 164 principal components (capturing 95% variance).

- **Experimental Results**

- Energy - based LMNN classification achieved a 1.4% test error rate, cutting the baseline kNN error rate by over one - third.
- Compared with other benchmarks (no extra prior knowledge), multi - layer neural nets had a 1.6
- The error rate of LMNN could be further reduced by learning different distance metrics for each digit class.

- **Experimental Results**

- LMNN showed good performance on the MNIST dataset, improving the accuracy of handwritten digit recognition.

Handwritten Digit Recognition Results

- **Dataset Description**

- MNIST: widely used for handwritten digit recognition.
- Images deskewed, projected to 164 PCs (95% variance).

- **Experimental Results**

- Energy - based LMNN: 1.4% test error, cut kNN error by over 1/3.
- Compared: multi - layer neural nets (1.6%), SVMs (1.2%).
- LMNN error can be further reduced with per - class metrics.

- **Experimental Results**

- LMNN performed well, close to multi - layer neural nets.
- Per - class metrics offer potential for further improvement.

Handwritten Digit Recognition Results

Benchmark test error rates

statistics	mnist	letters	20news	isolet	yFaces	bal	oFaces	wine	iris
# inputs	70000	20000	18827	7797	2414	535	400	152	128
# features	784	16	20000	617	8064	4	200	13	4
# reduced dimensions	164	16	200	172	300	4	200	13	4
# training examples	60000	14000	13179	6238	1690	375	280	106	90
# testing examples	10000	6000	5648	1559	724	161	120	46	38
# classes	10	26	20	26	38	3	40	3	3
# of train/test splits	1	10	10	1	10	100	100	100	100
% validation	0	0	0	0	30	0	30	0	30
kNN									
Euclidean	2.12	4.68	48.57	8.98	29.19	18.33	6.03	25.00	4.87
PCA	2.43	4.68	18.22	8.60	10.79	18.33	2.80	25.00	4.87
LDA	6.16	4.63	16.15	5.90	4.80	10.82	10.01	2.17	4.00
RCA	5.93	4.34	16.06	5.71	4.83	12.31	10.02	2.28	3.71
MCC	N/A	N/A	N/A	N/A	N/A	15.66	15.91	30.96	3.55
NCA	N/A	N/A	N/A	N/A	N/A	5.33	2.60	28.67	4.32
LMNN									
PCA	1.72	3.60	14.98	4.36	5.90	11.16	3.28	8.72	4.37
LDA	6.16	3.61	16.98	5.84	5.08	10.84	40.72	2.11	3.79
LMNN (energy)	1.37	2.67	22.09	3.40	10.11	9.14	3.16	7.67	3.68
LMNN (multiple passes)	1.69	2.80	13.83	4.30	5.52	5.86	4.83	7.59	4.26
LMNN (multiple metrics)	1.18	3.06	12.82	4.04	4.05	10.72	3.11	8.72	4.66
solver statistics									
CPU time (1M)	3h 25m	2m	70m	20m	8m	6s	66s	14s	2s
CPU time (MM)	8h 43m	14m	74m	84m	14m	8s	149s	16s	5s
# active constraints (1M)	540037	135715	676482	64396	86994	41522	3843	10194	574
# active constraints (MM)	305114	18588	101803	135832	39135	31717	70	748	1548
multiclass SVM									
	1.20	3.21	8.04	3.40	15.22	1.92	1.90	22.24	3.45

larger data sets

smaller data sets

Figure: Results and statistics of all experiments are presented. Datasets are ordered from largest to smallest. The table lists data stats and error rates of various LMNN training (single-pass, multi-pass, multi-metric), testing (kNN, energy-based), and preprocessing (PCA, LDA) methods. Results of RCA, NCA, and SVMs are included for comparison.

Handwritten Digit Recognition Results

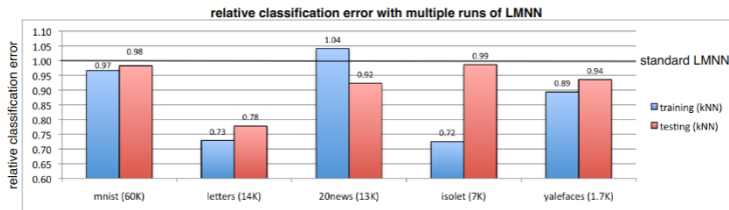


Figure: The relative change of the 3 – NN classification error after multiple runs of LMNN over a single run of LMNN.

References

-  Kilian Q. Weinberger, Lawrence K. Saul, "Distance Metric Learning for Large Margin Nearest Neighbor Classification", Journal of Machine Learning Research 10 (2009) 207 - 244
-  T. Cover, P. Hart, "Nearest neighbor pattern classification", IEEE Transactions in Information Theory, IT - 13, 21 - 27 (1967)
-  S. Belongie, J. Malik, J. Puzicha, "Shape matching and object recognition using shape contexts", IEEE Transactions on Pattern Analysis and Machine Intelligence, 24(4), 509 - 522 (2002)
-  P. Y. Simard, Y. LeCun, J. Decker, "Efficient pattern recognition using a new transformation distance", in Advances in Neural Information Processing Systems 6, 50 - 58 (1993)